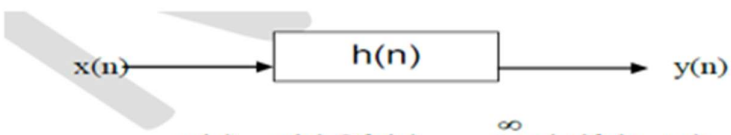
Linear and Circular Convolution

Requirement :- MATLAB Software, Code Composer Studio and DSP Kit

Theory:- Convolution is operated between two signals and includes the following operations: Folding, Shifting, Multiplication, and Addition.
Convolution sum is useful to obtain the response of discrete LTI system with input x(n) and impulse response h(n).



$$y(n) = x(n) \otimes h(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m)$$

DFT does not support linear convolution, because of x(n) and h(n) are of different durations. That's why we can go for circular convolution.
DFT supports only circular convolution, because of x(n) and h(n) are equal durations.
Circular convoluted sequence y(n) between x(n) and h(n) with equal duration N is defined as

$$y(n) = \sum_{m=0}^{N-1} x(m)h(n-m)$$

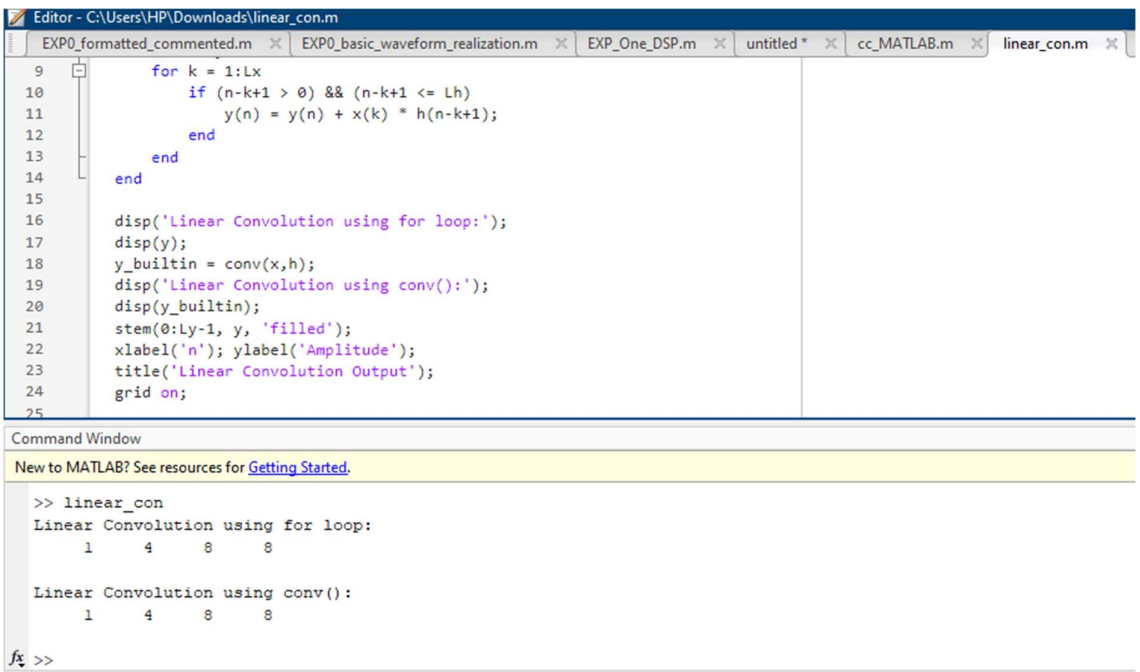
<u>Aim 1</u>: To compute the response of a discrete LTI system within put sequence x(n) and impulse response h(n) by using linear convolution without using the inbuilt conv function. Verify your program using the conv function for sequences x = [1, 2] and h = [1, 2, 4].

Code :-

```
x = [1 2];

h = [1 2 4];
Lx = length(x);
```

```
Lh = length(h);
Ly = Lx + Lh - 1;
y = zeros(1, Ly);

for n = 1:Ly
    for k = 1:Lx
        if (n-k+1 > 0) && (n-k+1 <= Lh)
            y(n) = y(n) + x(k) * h(n-k+1);
        end
    end
end

disp('Linear Convolution using for loop:');
disp(y);
y_builtin = conv(x,h);
disp('Linear Convolution using conv():');
disp(y_builtin);
stem(0:Ly-1, y, 'filled');
xlabel('n'); ylabel('Amplitude');
title('Linear Convolution Output');
grid on;
```

Figure:-



Conclusion:-

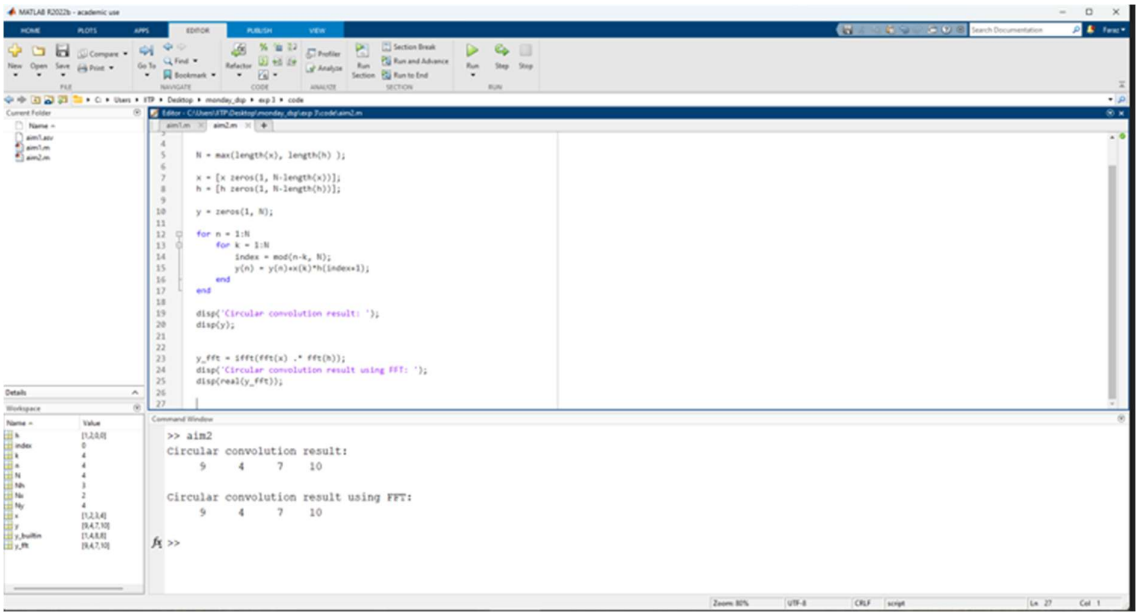
In this experiment, the response of a discrete-time LTI system was computed using linear convolution implemented without the inbuilt conv() function. The convolution operation was carried out by explicitly performing time shifting, sample-by-sample multiplication, and summation, which represent the fundamental behavior of LTI systems. For the given input sequence (x(n) = [1 2]) and impulse response (h(n) = [1 2 4]), the length of the output sequence was correctly obtained as (N = $N_x + N_h$-1). The result produced by the manually written convolution program exactly matched the output obtained using MATLAB's inbuilt conv() function, thereby verifying the correctness of the algorithm. This experiment confirms the theoretical concept of linear convolution and reinforces the linearity and time-invariance properties of discrete-time LTI systems.

<u>Aim 2</u>: Write a program in MATLAB to compute the response of a discrete LTI system with input sequence x(n) and impulse response h(n) by using Circular convolution. Check your result for the sequences x(n) = [1,2,3,4] and h(n) = [1,2].

Code:-

```
x = [1 2 3 4];
h = [1 2 ];
N = max(length(x), length(h));
x = [x zeros(1, N-length(x))];
h = [h zeros(1, N-length(h))];
y = zeros(1, N);

for n = 1:N
    for k = 1:N
        index = mod(n-k, N);
        y(n) = y(n)+x(k)*h(index+1);
    end
end

disp('Circular convolution result: ');
disp(y);


y_cconv = cconv(x, h, 4);
disp('Circular convolution result using FFT: ');
disp(real(y_cconv));
```

Figure :-



Conclusion:-

In this experiment, the response of a discrete-time LTI system was computed using **circular convolution** for the given input sequence (x(n) = [1 2 3 4]) and impulse response (h(n) = [1 2]). The sequences were first made equal in length by appropriate **zero padding**, and circular convolution was then performed by applying the periodic time-indexing property. The result obtained from the circular convolution program was verified using an FFT-based method and found to be identical, confirming the correctness of the implementation. This experiment demonstrates how circular convolution is efficiently used in digital signal processing, especially in frequency-domain analysis and block-based signal processing techniques.

<u>Aim 3</u>: To write a "C" program to verify Linear Convolution and Circular Convolution graphically with using Code Composer Studio (CCS-3.1) IDE and the DSP TMS 320C6713 DSK Kit.

Code :-

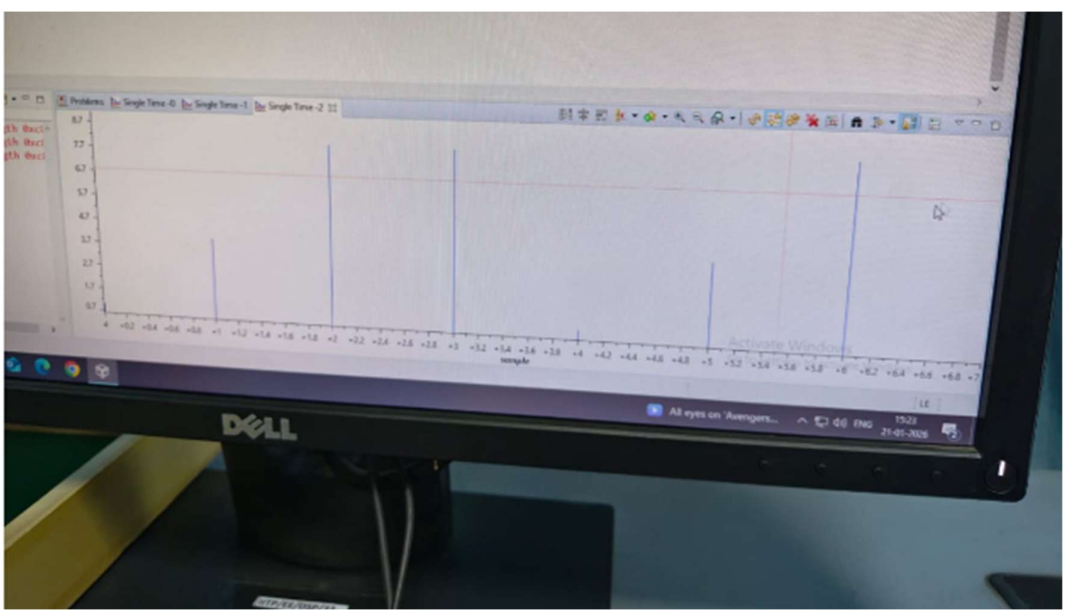
```
#include <stdio.h>

#define NX 4
#define NH 2
#define NY (NX + NH - 1)
#define N 4                    // Length for circular convolution

// Input sequences
float x[NX] = {1, 2, 3, 4};
float h[NH] = {1, 2};

// Output arrays
float y_linear[NY];
float y_circular[N];

void linear_convolution();
void circular_convolution();

void main()
{
   int i;
```

```
    // Initialize output arrays
    for(i = 0; i < NY; i++)
        y_linear[i] = 0;

    for(i = 0; i < N; i++)
        y_circular[i] = 0;

    linear_convolution();
    circular_convolution();

    while(1);
}

void linear_convolution()
{
    int n, k;

    for(n = 0; n < NY; n++)
    {
        for(k = 0; k < NX; k++)
        {
            if((n - k) >= 0 && (n - k) < NH)
            {
                y_linear[n] += x[k] * h[n - k];
            } } } }
```

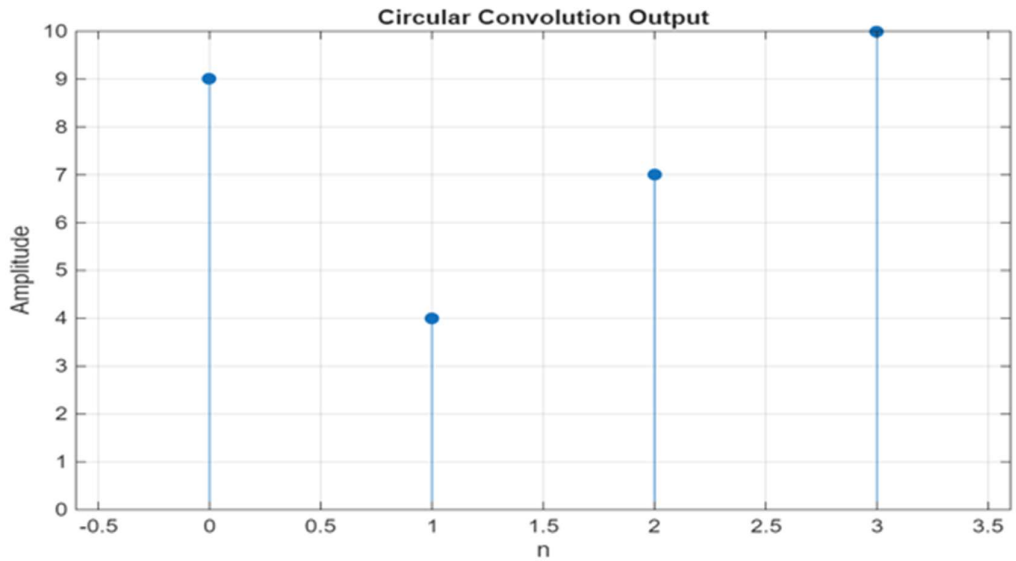
```
void circular_convolution()
{
  int n, k;
  float h_pad[N] = {1, 2, 0, 0};

  for(n = 0; n < N; n++)
  {
    for(k = 0; k < N; k++)
    {
      y_circular[n] += x[k] * h_pad[(n - k + N) % N];
    } } }
```

Figure :-

1. Linear Convolution :-



2. Circular Convolution :-



Conclusion :-

In this experiment, Linear Convolution and Circular Convolution were successfully implemented and verified using a **C program on the DSP Kit** with the **Code Composer Studio (CCS-3.1)** IDE. The convolution algorithms were executed on real DSP hardware, and the output sequences were visualized graphically using the CCS time-domain plotting tools. The linear convolution output correctly represented the actual response of a discrete LTI system, while the circular convolution output demonstrated the periodic convolution behavior used in frequency-domain and FFT-based signal processing. The graphical verification confirmed that the practical DSP results matched theoretical expectations, thereby validating both convolution techniques and providing hands-on understanding of real-time digital signal processing implementation on the TMS320C6713 platform.